



Compilador Koi

Compilador x86 para el lenguaje Carp

Curso y sección

Compiladores — Teoría 1

Integrantes

Yeimi Adelmara Varela Villarreal 202410586

Tiziano Abraham Lopez Vargas 202310267

Alessandra Valeria Silva Rios 202310338

Profesor

Julio Eduardo Yarasca Moscol

Julio 2026

Índice

1. Introducción	1
1.1. Antecedentes del curso	1
1.2. ¿Por qué Carp?	1
1.3. Objetivos del proyecto	2
2. Arquitectura del sistema	2
2.1. Visión general del pipeline	2
2.2. IDE Pond	3
2.3. Justificación de la arquitectura de tres procesos	4
2.4. Tamaño y organización del código fuente	4
3. Implementación técnica	5
3.1. Análisis léxico	5
3.2. Análisis sintáctico y construcción del AST	5
3.3. Análisis de ámbito (<i>scope</i>)	7
3.4. Sistema de tipos e inferencia	8
3.4.1. Representación de tipos	8
3.4.2. Generación de restricciones	8
3.4.3. Unificación de Robinson	8
3.4.4. Un hallazgo arquitectónico central: ausencia de generalización <i>let-polymorphism</i>	9
3.5. Monomorphización	9
3.6. Conversión de clausuras (<i>closure conversion</i>)	10
3.7. Generación de representación intermedia en forma SSA	11
3.7.1. <code>if</code> como expresión: instantánea y fusión	12
3.7.2. <code>loop</code> y <code>while</code> : cabeceras con <code>Phi</code> y arcos de retroceso	12
3.8. Evidencias del IDE Python	12
4. Extensión del lenguaje	15
5. Optimización	17

5.1. Optimización a nivel de representación intermedia	17
5.2. Optimización de mirilla sobre el ensamblador emitido	18
5.3. Cobertura de pruebas de las optimizaciones	18
6. Mejoras y limitaciones	19
7. Pruebas	20
7.1. Metodología de pruebas	20
7.2. Cobertura de pruebas automatizadas	20
7.3. Programas de prueba de extremo a extremo	21
8. Resultados de <i>benchmark</i>	22
8.1. Metodología	22
8.2. Resultados	22
8.3. Análisis de resultados	24
9. Conclusiones	25
Referencias	26

1. Introducción

1.1. Antecedentes del curso

El curso de Compiladores aborda, de manera progresiva, las fases clásicas que transforman un programa escrito en un lenguaje de alto nivel en código ejecutable por una arquitectura concreta: análisis léxico mediante autómatas finitos, análisis sintáctico mediante gramáticas libres de contexto mejor implementadas con parsers predictivos o descendentes, análisis semántico apoyado en tablas de símbolos y sistemas de tipos, generación de una representación intermedia adecuada para razonar sobre el programa independientemente de la arquitectura destino, y finalmente generación de código de máquina sujeta a las restricciones de una arquitectura y una convención de llamada (*ABI*) reales. El presente proyecto exige integrar estas fases en un compilador funcional, aplicar al menos un conjunto de optimizaciones con evidencia medible, y contrastar el resultado contra compiladores de uso extendido en la industria, en línea con la práctica de evaluación experimental esperada en un curso de sistemas de software complejos.

Este informe documenta el diseño, la implementación y la evaluación de **Koi**, el compilador desarrollado para satisfacer dicho proyecto. Adicionalmente, el trabajo se complementa con **Pond**, un entorno TUI en Python que actúa como interfaz de edición, selección de versiones y ejecución del pipeline del compilador.

1.2. ¿Por qué Carp?

El equipo eligió como lenguaje fuente para elaborar este compilador, un subconjunto de **Carp**; un lenguaje funcional de la familia Lisp basado en *S-expressions*, con tipado estático, inferencia de tipos automática y un sistema de memoria sin recolector de basura. La elección respondió a tres consideraciones técnicas concretas:

- (a) **Una gramática de superficie mínima simplifica el análisis sintáctico sin trivializar el resto del compilador.** La sintaxis de S-expressions (listas totalmente parentizadas, sin precedencia de operadores ni ambigüedades de gramática) permite implementar un parser *recursivo-descendente* de un único token de *lookahead* (LL(1)) sin necesidad de resolver conflictos de *shift/reduce* ni de mantener una tabla de precedencia de operadores.
- (b) **Carp exige, en su propia definición, las características avanzadas de un lenguaje de programación de alto nivel.** Sin macros homoicónicas de por medio, un subconjunto realista de Carp igual requiere resolver: inferencia de tipos al estilo Hindley–Milner, punteros explícitos y gestión manual de memoria (`malloc/free`,

sin recolector de basura como en Rust o como runtime propio como en Go), tipos genéricos vía monomorphización, y funciones de primera clase con *closures* reales (conversión de clausuras).

- (c) **Carp es suficientemente distinto de C/Rust/Go como para que la comparación experimental sea informativa.** Al no heredar sintaxis ni convenciones de la familia C, cualquier similitud de rendimiento frente a Rust o Go con un backend igual de simple que el de Koi es evidencia genuina de las decisiones de diseño del compilador, no un artefacto de compartir infraestructura de bajo nivel con el lenguaje de comparación.

1.3. Objetivos del proyecto

Los objetivos específicos del presente proyecto se basan principalmente en:

- (a) Implementar las fases de análisis léxico, sintáctico y semántico del lenguaje Carp correspondientes a un compilador x86
- (b) Verificar verificación de tipos y manejo de errores léxicos, sintáctico y semántico
- (c) Generar código ensamblador x86-64 funcional
- (d) Aplicar técnicas de optimización con mejoras medibles
- (e) Desarrollar un conjunto de benchmark para realizar una comparación experimental del compilador con el de Rust y Go

2. Arquitectura del sistema

2.1. Visión general del pipeline

Koi está implementado como un *workspace* de Cargo con tres crates de Rust totalmente independientes (*koi-ast*, *koi-ir* y *koi-assembly*), cada una compilada a un binario ejecutable propio. A diferencia de un compilador monolítico donde las fases se comunican mediante llamadas a función dentro del mismo proceso, en Koi cada etapa del pipeline se ejecuta como un proceso independiente que lee su entrada y escribe su salida como un archivo JSON en disco (*/tmp*). La Figura 1 resume el flujo completo, desde el código fuente en Carp hasta el ejecutable nativo.

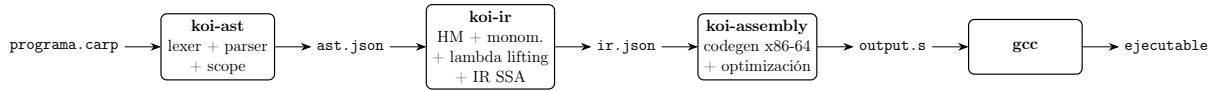


Figura 1: Pipeline de compilación de Koi. Cada recuadro es un binario de Rust independiente; cada elemento en fuente monoespaciada entre recuadros es un artefacto en disco que actúa como contrato de interfaz entre etapas.

Cada binario tiene una responsabilidad delimitada, resumida en la Tabla 1.

Cuadro 1: Responsabilidad de cada crate del workspace.

Crate	Entrada	Salida	Responsabilidad
koi-ast	programa.carp	ast.json	Análisis léxico, análisis sintáctico, construcción del AST, análisis de ámbito (<i>scope</i>).
koi-ir	ast.json	ir.json	Generación de restricciones de tipo e inferencia Hindley–Milner, monomorphización, conversión de clausuras (<i>lambda lifting</i>), generación de representación intermedia en forma SSA.
koi-assembly	ir.json	output.s	Asignación de posiciones de memoria, generación de código x86-64 AT&T, optimización a nivel de IR, optimización de mirilla (<i>peephole</i>) sobre el ensamblador emitido.

2.2. IDE Pond

El proyecto está dividido en dos repositorios con responsabilidades distintas. Mientras Koi implementa el compilador propiamente dicho, Pond aporta la interfaz de usuario y la orquestación del flujo de compilación. Su código está escrito en Python y usa `Textual` para ofrecer un IDE en terminal con editor, panel de salida y acciones de compilación/ejecución.

En términos de arquitectura, Pond se organiza en cinco módulos principales: `main.py` como punto de entrada; `ui.py` para la interfaz TUI; `compiler.py` como envoltura del pipeline `koi-ast` $\rightarrow$ `koi-ir` $\rightarrow$ `koi-assembly` $\rightarrow$ `gcc`; `manager.py` para gestión de versiones y descarga de binarios; y `syntax_highlighter.py` para el editor con resaltado de sintaxis de Carp. Pond no reemplaza ninguna fase del compilador: su función es exponerlas de forma utilizable y verificable desde una sola aplicación.

2.3. Justificación de la arquitectura de tres procesos

La decisión de comunicar las tres fases mediante archivos JSON en disco, en lugar de estructuras de datos compartidas en memoria dentro de un único proceso, fue deliberada y respondió a una necesidad organizativa concreta del equipo. Al fijar el contrato de interfaz como un esquema JSON versionado por convención de nombres, cada interfaz podía iterar sobre su propia crate sin necesidad de que el código de las otras fases compilara en todo momento.

Esta arquitectura tiene, sin embargo, dos costos que se hicieron evidentes durante el desarrollo. Primero, cualquier cambio de esquema en un extremo del contrato rompe el pipeline de manera completamente silenciosa hasta que se ejecuta todo el flujo. Segundo, cualquier información no serializada explícitamente en el JSON se pierde irrecuperablemente entre etapas. Tercero, las posiciones de línea y columna de los nodos del AST, cruciales para reportar errores con ubicación exacta, no sobreviven la conversión a instrucciones de IR, y por lo tanto los errores que pudiera reportar la fase de generación de código carecen de esa información en la práctica.

2.4. Tamaño y organización del código fuente

El proyecto completo suma 29 módulos en total, donde Koi reúne 24 módulos Rust distribuidos entre sus tres crates y Pond añade 5 módulos Python de aplicación. La crate `koi-ir` concentra por sí sola cerca del 54 % del código total, proporción coherente con que allí reside la parte conceptualmente más compleja del compilador: el sistema de inferencia de tipos, la conversión de clausuras y la construcción de la representación intermedia en forma SSA.

Cuadro 2: Organización del código fuente principal por repositorio/componente.

Componente	Módulos principales
<code>koi-ast</code>	<code>token.rs</code> , <code>scanner.rs</code> , <code>parser.rs</code> , <code>ast.rs</code> , <code>scope.rs</code> , <code>main.rs</code>
<code>koi-ir</code>	<code>types.rs</code> , <code>inference.rs</code> , <code>unification.rs</code> , <code>monomorphizer.rs</code> , <code>lambda_lifter.rs</code> , <code>ir.rs</code> , <code>ir_generator.rs</code> , <code>builtins.rs</code> , <code>pipeline.rs</code> , <code>ast.rs</code> , <code>main.rs</code>
<code>koi-assembly</code>	<code>ir_parser.rs</code> , <code>register_allocator.rs</code> , <code>codegen.rs</code> , <code>optimizer.rs</code> , <code>abi.rs</code> , <code>main.rs</code>
<code>Pond</code>	<code>main.py</code> , <code>ui.py</code> , <code>compiler.py</code> , <code>manager.py</code> , <code>syntax_highlighter.py</code>

En cuanto al estilo de implementación, las tres crates siguen consistentemente el idioma de *enums algebraicos etiquetados + pattern matching exhaustivo*, en lugar de un patrón *visitor* orientado a objetos; decisión de diseño estándar en compiladores escritos en len-

guajes con tipos suma de primera clase (Rust, OCaml, Haskell), que además obliga al compilador de Rust a verificar en tiempo de compilación que ninguna variante de un nodo del AST o de una instrucción de IR quede sin manejar en ningún punto del pipeline.

3. Implementación técnica

Esta sección documenta, fase por fase, cómo se implementó cada etapa clásica de compilación, vinculando la teoría del curso con las decisiones concretas tomadas en el código.

3.1. Análisis léxico

El escáner (`koi-ast/src/scanner.rs`) recorre el código fuente carácter a carácter sobre un vector de caracteres, manteniendo en todo momento la posición actual como par (`line`, `column`), incrementada en cada avance y reiniciada al cruzar un salto de línea, cuya información es indispensable para que las fases posteriores puedan reportar errores con ubicación exacta.

El reconocimiento de tokens sigue las reglas léxicas típicas de un dialecto de Lisp: los comentarios inician con el carácter `;` y se descartan íntegramente hasta el fin de línea sin generar ningún token; las cadenas de texto se delimitan con comillas dobles y soportan las secuencias de escape estándar (`\n \t \r \\ \"`); los literales numéricos se reconocen como una corrida de dígitos que se clasifica como número de punto flotante únicamente si el carácter `.` viene seguido de otro dígito (evitando así que un punto aislado a fin de token se interprete erróneamente como parte del número); y los identificadores siguen la expresión regular `[a-zA-Z_][a-zA-Z0-9_?!-]*`, donde el guion medio está permitido en cualquier posición intermedia, habilitando la convención *kebab-case* típica de Lisp (`apply-func`, `set-field!`).

3.2. Análisis sintáctico y construcción del AST

El analizador sintáctico (`koi-ast/src/parser.rs`) implementa un parser **recursivo-descendente predictivo LL(1)**, es decir, con exactamente un token de *lookahead* (`current : TokenWithPos`) y sin retroceso (*backtracking*). La elección es consistente con la sintaxis totalmente parentizada del lenguaje fuente: dado que cada forma sintáctica comienza con un paréntesis seguido de una palabra clave o un símbolo identificable en una sola mirada, la gramática de Carp es LL(1) por construcción y no requiere resolución de ambigüedades ni tablas de precedencia de operadores.

El punto de entrada al análisis de cualquier expresión, `parse_paren_body`, es compartido entre el nivel superior del programa y cualquier posición interna de expresión, de modo que todas las formas especiales del lenguaje son válidas tanto en la parte más externa del programa como anidadas arbitrariamente dentro de otras expresiones. La Tabla 3 resume la gramática de formas especiales reconocidas por el parser.

Cuadro 3: Formas especiales reconocidas por el parser de Koi.

Forma	Sintaxis concreta	Observación
<code>defn</code>	<code>(defn nombre [p1 p2:tipo ...] cuerpo)</code>	solo a nivel de programa
<code>defstruct</code>	<code>(defstruct Nombre [f1 t1] [f2 t2] ...)</code>	solo a nivel de programa
<code>if</code>	<code>(if cond then [else])</code>	rama <code>else</code> opcional
<code>let</code>	<code>(let [n1 v1 n2 v2 ...] cuerpo)</code>	secuencial (semántica <code>let*</code>)
<code>loop</code>	<code>(loop [var init] cond step cuerpo)</code>	un único índice mutable, estilo <code>for</code> de C
<code>while</code>	<code>(while cond cuerpo)</code>	mutación arbitraria vía <code>set!</code>
<code>lambda</code>	<code>(lambda [p1 p2 ...] cuerpo)</code>	función anónima
<code>field</code>	<code>(field obj nombreCampo)</code>	el nombre de campo es un símbolo literal, no una expresión
<code>set-field!</code>	<code>(set-field! obj nombreCampo valor)</code>	escritura de campo de struct
<code>set!</code>	<code>(set! nombre valor)</code>	mutación de un <i>binding</i> local
<code>index</code>	<code>(index arr idx)</code>	lectura de arreglo
<code>new</code>	<code>(new TipoNombre [tam-o-init])</code>	reserva de memoria (<i>struct</i> o arreglo)
<code>do</code>	<code>(do e1 e2 ... en)</code>	secuencia de $n \geq 1$ expresiones
—	<code>(f a1 a2 ...)</code>	llamada genérica (caso por defecto)
—	<code>&expr</code>	dirección-de (<i>address-of</i>)
—	<code>*expr</code>	desreferencia; <code>*</code> como cabeza de llamada es multiplicación
—	<code>[e1 e2 ...]</code>	literal de arreglo

El árbol de sintaxis abstracta se define en `koi-ast/src/ast.rs` como un *enum* de Rust con la anotación `#[serde(tag = "nodeType")]`, que produce automáticamente, vía la biblioteca `serde`, una serialización JSON etiquetada. El tipo `ASTNode` define veinte variantes (`Program`, `FunctionDef`, `StructDef`, `Call`, `Variable`, `Literal`, `Lambda`, `LetBinding`,

`IfExpr`, `LoopExpr`, `FieldAccess`, `SetField`, `Index`, `AddrOf`, `Deref`, `New`, `ArrayLiteral`, `SetVar`, `WhileExpr`, `DoExpr`), cada una portando su propia posición de línea y columna de origen para diagnóstico.

Manejo de errores sintácticos. El parser implementa un mecanismo de reporte de error consistente, aunque sin recuperación: ante la primera producción que no coincide con lo esperado, el análisis se aborta de inmediato (propagación mediante el operador `?` de Rust sobre valores `Result`) y se produce un mensaje con el formato:

```
{mensaje}. Got {token} at line {L}, column {C}
Unexpected token: {token} at line {L}, column {C}
```

El binario `koi-ast` antepone el prefijo `[parser]` al mensaje antes de escribirlo en la salida de error estándar y terminar con código de salida 1.

3.3. Análisis de ámbito (*scope*)

El analizador de ámbito (`koi-ast/src/scope.rs`) implementa una pila de tablas de símbolos (`Vec<HashMap<String, String>`), usada efectivamente como una pila de conjuntos) y opera en dos pasadas. La primera pasada (`collect_top_level`) pre-registra todos los nombres de funciones y *structs* declarados a nivel de programa antes de analizar ningún cuerpo, lo que habilita referencias hacia adelante y recursión mutua entre funciones. La segunda pasada recorre el árbol completo verificando que toda variable leída (en posición de `Variable` o como destino de `SetVar`) haya sido declarada previamente, sea el nombre de una función de programa, o corresponda a un operador *built-in* del lenguaje.

Es importante precisar el alcance real de esta fase para no sobreestimarla: el análisis de ámbito *únicamente* detecta el uso de variables no declaradas. No implementa detección de *shadowing* no intencional (rebindear un nombre dentro de un `let` anidado es válido y se trata como sombreado intencional), ni detección de declaraciones duplicadas de funciones o *structs* a nivel de programa (una segunda declaración con el mismo nombre simplemente sobrescribe silenciosamente el registro anterior). Los nombres de campos de *struct* tampoco se verifican en esta fase, ya que en la sintaxis del lenguaje son símbolos literales y no expresiones evaluables. Los errores encontrados se acumulan en un vector en lugar de abortar en el primer hallazgo, y se reportan todos juntos al finalizar el recorrido completo del árbol (una estrategia de recuperación de errores más amigable para el usuario que la del parser, que sí es *fail-fast*.)

3.4. Sistema de tipos e inferencia

El sistema de tipos de Koi está distribuido en tres módulos de `koi-ir`: la representación de tipos (`types.rs`), la generación de restricciones (`inference.rs`) y el algoritmo de unificación (`unification.rs`). El diseño sigue de cerca el esquema clásico del *Algorithm W* de Hindley–Milner: primero se recorre el árbol de sintaxis generando un conjunto de restricciones de igualdad entre tipos, sin resolver nada todavía, y solo después se resuelve ese conjunto completo mediante unificación de Robinson.

3.4.1. Representación de tipos

El tipo `Type` es un enum cerrado con las variantes `Int64`, `Float64`, `Bool`, `String`, `Array` (`Box<Type>`), `Pointer(Box<Type>)`, `Struct(String)`, `Function { params, return_type }`, `Variable(TypeVar)` y `Unit` (este último, el tipo de expresiones evaluadas solo por su efecto, como `set!` o `while`, que no producen ningún valor útil). Cada variable de tipo fresca se obtiene mediante un contador global atómico (`AtomicUsize`), evitando el uso de código `unsafe` que exigiría un contador mutable estático tradicional.

3.4.2. Generación de restricciones

El generador de restricciones recorre el programa en tres pasadas: (1) registra los tipos resueltos de los campos de cada *struct* declarado, independientemente del orden de declaración; (2) registra la firma de cada función (los parámetros anotados explícitamente se resuelven directamente, los no anotados reciben una variable de tipo fresca), y el tipo de retorno *siempre* es una variable fresca, ya que la sintaxis de superficie de Carp/Koi no admite anotación de tipo de retorno; y (3) recorre el cuerpo de cada función generando una restricción por cada construcción del lenguaje que impone una relación de tipos. Por ejemplo, la condición de un `if` debe unificar con `Bool`, las dos ramas de un `if` deben unificar entre sí, el índice de un arreglo debe unificar con el elemento de un tipo `Array` fresco, y así sucesivamente para cada nodo del AST.

3.4.3. Unificación de Robinson

El módulo `unification.rs` implementa el algoritmo de unificación estructural de libro de texto: se recorre la lista de restricciones en orden, aplicando en cada paso la sustitución acumulada hasta el momento a ambos lados de la restricción antes de intentar unificarla, de modo que un enlace de tipo resuelto tempranamente afecta correctamente a las restricciones posteriores que lo mencionan. La función de enlace de una variable de tipo (`Substitution::bind`) ejecuta primero una verificación de ocurrencia (*occurs check*) para rechazar la construcción de un tipo infinito (por ejemplo, `T = Array(T)`), tal como

exige la corrección formal del algoritmo. Cuando la unificación falla, se produce un mensaje de error que incluye el contexto semántico legible en el que se originó la restricción (por ejemplo, “*if branches*” o “*aset! value must match element type*”), junto con la línea y columna de origen.

Una vez resuelta toda restricción posible, cualquier variable de tipo que permanezca sin enlazar (por ejemplo, la de un parámetro genuinamente no utilizado en el cuerpo de una función) se resuelve por defecto a `Int64` en lugar de producir un error.

3.4.4. Un hallazgo arquitectónico central: ausencia de generalización *let-polymorphism*

Las funciones de Koi nunca se generalizan a un esquema de tipos polimórfico. Cada declaración `defn` obtiene exactamente *una* firma de tipo `Function` global (un monotipo, no un tipo universalmente cuantificado) y cada sitio de llamada a esa función restringe sus argumentos contra esa misma firma compartida. En consecuencia, Koi implementa generación de restricciones y unificación al estilo *Algorithm W*, pero sin la regla de generalización que caracteriza a Hindley–Milner completo (la regla LET que introduce cuantificación universal al vincular una expresión a un nombre). El efecto observable es que una función genuinamente polimórfica como `(defn identity [x] x)`, si se invoca una vez con un argumento `i64` y otra vez con un argumento `string` dentro del mismo programa, falla al unificar en lugar de obtener dos instanciaciones de tipo independientes. Esta observación es central para interpretar correctamente el estado del mecanismo de monomorphización explicado a continuación.

3.5. Monomorphización

El módulo `monomorphizer.rs` implementa, de manera completa y con su propia batería de pruebas unitarias, el mecanismo estándar de especialización de funciones genéricas por combinación de tipos de argumento: registra cada tupla de tipos con la que se invoca una función (`record_call`), genera un nombre de función único por combinación mediante *name mangling* (`mangle_name`, con el esquema “`{base}__{tipo1}_{tipo2}...`”), y produce una copia especializada y renombrada de la definición original por cada combinación distinta observada.

Dado que el sistema de tipos no generaliza funciones, cualquier función que en la práctica se hubiese invocado con dos tuplas de tipo distintas ya habría hecho fallar la unificación (y por lo tanto abortado la compilación) antes de que el pipeline alcanzara siquiera la etapa de monomorphización. La conclusión honesta, verificada contra el propio código, es que `specialize_program` es un mecanismo funcionalmente correcto pero que constituye una operación identidad (*no-op*) sobre cualquier programa `.carp` real compi-

lado por Koi; el mecanismo únicamente se ejercita en las pruebas unitarias del módulo, que invocan `record_call` directamente con datos sintéticos de múltiples tuplas. El andamiaje de monomorphización existe, está correctamente implementado de forma aislada y probada, pero el sistema de tipos tal como está conectado hoy en el pipeline completo no llega a producir funciones polimórficas que requieran especializarse en la práctica.

3.6. Conversión de clausuras (*closure conversion*)

La conversión de *lambdas* a funciones de primer orden con entorno explícito (técnica estándar para compilar funciones de orden superior a una arquitectura sin soporte nativo para funciones anidadas) se implementa en `lambda_lifter.rs`. El algoritmo recorre recursivamente cada nodo `Lambda` del AST, procesando primero el cuerpo de la lambda (de modo que cualquier *lambda* anidada quede ya resuelta antes de analizar las variables libres de la actual), y calcula el conjunto de variables libres mediante un recorrido estructural estándar que distingue entre nombres ligados (parámetros, variables de `let/loop` en el ámbito actual) y nombres globales (funciones de programa y operadores *built-in*): todo nombre que no pertenezca a ninguno de los dos conjuntos se marca como variable libre.

Cuando una *lambda* no tiene ninguna variable libre, se eleva directamente a una función de programa ordinaria (renombrada `_lambda_N`), sin necesidad de ningún mecanismo de entorno; el nodo `Lambda` original se reemplaza simplemente por una referencia a esa función. Cuando sí existen variables libres, se genera un tipo *struct* sintético de entorno con un campo por cada variable capturada, se antepone ese entorno como parámetro implícito de la función elevada, cada referencia a una variable libre dentro del cuerpo se reescribe como un acceso de campo sobre el entorno, y el nodo `Lambda` original se sustituye por un nodo interno `MakeClosure` que *nunca* atraviesa la frontera JSON hacia otra crate, únicamente lo produce el propio módulo de *lambda lifting* y lo consume el generador de IR dentro de la misma crate `koi-ir`.

En tiempo de ejecución, una clausura se representa como un puntero a un *struct* compartido de dos campos, `{fn_ptr, env_ptr}` (el mismo *struct* `Closure` para todas las clausuras del programa), donde `env_ptr` apunta a un *struct* particular de la clausura concreta con un campo por variable capturada, con su tipo real. Esta es la representación conocida como *fat pointer* o *closure de dos punteros*, análoga en espíritu a la que usan implementaciones de lenguajes funcionales sin recolector de basura dedicado a clausuras.

Limitaciones del mecanismo de clausuras. Se identificaron tres límites concretos, dos de diseño deliberado y uno descubierto durante la verificación de extremo a extremo:

- **Sin clausuras anidadas.** El mecanismo de reescritura de accesos a variables libres opera sobre una lista de *nombres* capturados, no sobre subárboles del AST, por

lo que no existe un punto donde reescribir el nombre capturado por una *lambda* exterior si ese nombre es a su vez una variable libre de una *lambda* interior que también captura. Una clausura que contiene, en su propio cuerpo, otra clausura capturadora queda fuera de alcance por construcción del algoritmo.

- **Semántica de captura por valor, no por referencia.** Una clausura toma una *snapshot* del valor de la variable capturada en el momento exacto de su construcción. Si esa variable se muta posteriormente mediante `set!`, la clausura ya construida no observa el cambio. Es una decisión de diseño razonable, puesto que evita problemas de *aliasing* sin necesidad de implementar un *borrow checker* completo, en el mismo espíritu que el sistema de *ownership* de Carp real, pero no está formalmente documentada como garantía del lenguaje más allá de este informe.
- **Clausuras pasadas como parámetro de función genérica producen una falla de segmento.** Una clausura creada e invocada dentro de la misma función funciona correctamente en cualquier contexto verificado. Sin embargo, pasarla como argumento a una función reutilizable que la invoque genéricamente (el patrón típico de un `map` o `filter` de orden superior) produce un *segfault* en tiempo de ejecución: la marca de tipo sintética que identifica un valor como clausura (necesaria para decidir si hay que desempaquetar `fn_ptr/env_ptr` antes de invocar) solo se asigna en el sitio exacto donde se construye la clausura, y no se propaga a través de la firma de tipo de una función que la recibe como parámetro. La consecuencia práctica, verificada durante la construcción del benchmark `lambda_map` (§8), es que no es posible escribir un `map/filter/reduce` genérico y reutilizable sobre clausuras en Koi; cada uso concreto debe integrar (*inline*) su propio bucle en la misma función donde la clausura se construye.

3.7. Generación de representación intermedia en forma SSA

El generador de IR (`koi-ir/src/ir_generator.rs`) baja el AST, ya libre de clausuras de orden superior gracias a la fase anterior, a una representación intermedia explícitamente en **forma SSA** (*Static Single Assignment*): cada valor calculado recibe un nombre único (`%v0`, `%v1`, ...) que se asigna exactamente una vez, y el flujo de control se organiza en bloques básicos conectados mediante instrucciones de salto y de rama. El tipo `Instruction` es un enum de catorce variantes (`Const`, `BinOp`, `Call`, `Return`, `Jump`, `Branch`, `Phi`, `CallIndirect`, `Alloc`, `GetField`, `SetField`, `GetIndex`, `SetIndex`, `AddrOf`, `Deref`).

La instrucción `Phi` merece atención particular porque es la pieza que permite representar, dentro de una forma puramente SSA, construcciones del lenguaje fuente que sí permiten que un valor dependa del camino de ejecución tomado. Típicamente, el resultado de un `if` usado como expresión, o el valor de una variable de control al salir de un

bucle. Un nodo `Phi` fusiona, en el punto donde convergen dos o más bloques predecesores, los distintos valores SSA que la misma variable lógica pudo haber tomado en cada camino, produciendo un único valor SSA nuevo válido a partir de ese punto.

3.7.1. `if` como expresión: instantánea y fusión

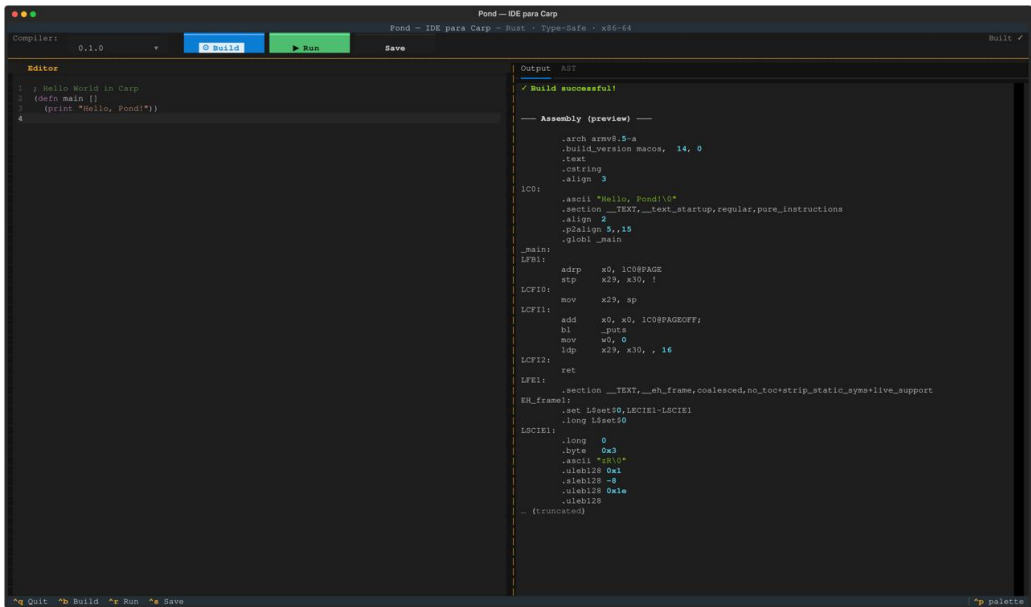
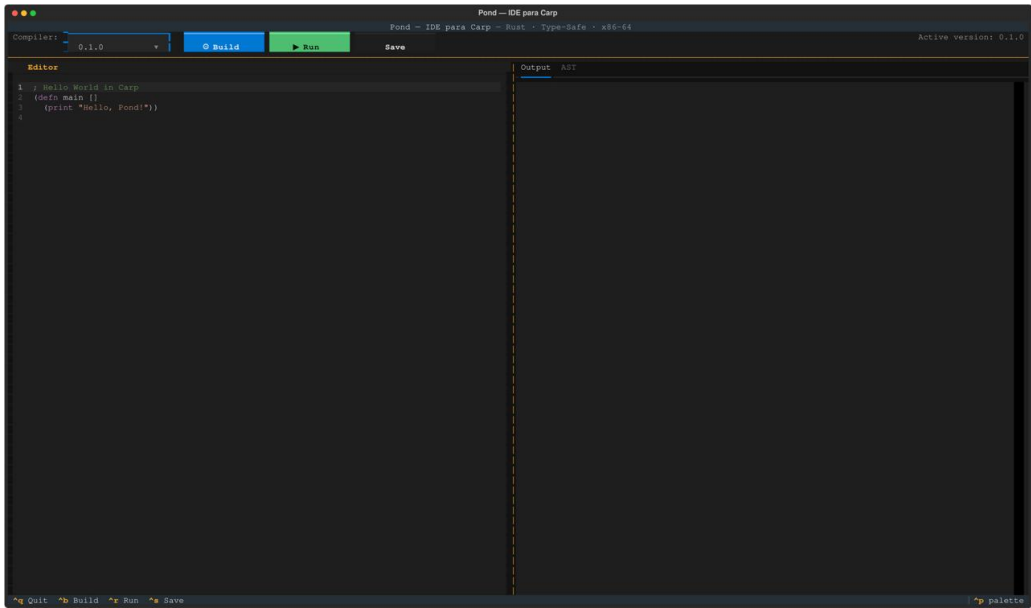
La función `generate_if` implementa el *lowering* clásico de un condicional usado como expresión: antes de generar la rama verdadera y la rama falsa, se toma una instantánea del estado SSA visible de cada variable en ámbito; ambas ramas se generan a partir de esa misma instantánea (de modo que la rama falsa nunca observa mutaciones hechas por la rama verdadera, y viceversa); y, tras generar ambas ramas, toda variable cuyo valor SSA final difiera entre los dos caminos recibe un nodo `Phi` en el bloque de fusión, quedando reasignada a ese resultado fusionado para el código posterior al condicional. Este mecanismo es precisamente lo que hace correcta una mutación con `set!` dentro de una sola rama de un `if`: sin él, el código posterior al condicional podría terminar leyendo un valor SSA definido únicamente en la rama que no se ejecutó en tiempo de ejecución.

3.7.2. `loop` y `while`: cabeceras con `Phi` y arcos de retroceso

La construcción original `loop` (con una única variable de control mutable, estilo bucle `for` de C) se traduce al patrón estándar de cabecera de bucle en SSA: el bloque de cabecera recibe un nodo `Phi` cuyo único arco de entrada conocido inicialmente es el valor previo al bucle; se genera el cuerpo del bucle; y, una vez conocido el valor final de la variable de control tras una iteración, ese valor se registra como el arco de retroceso (*back edge*) faltante del mismo `Phi`. El valor del `loop` como expresión es el resultado de ese `Phi` al salir, no la última expresión evaluada dentro del cuerpo.

La construcción `while`, agregada como extensión del lenguaje, generaliza exactamente esta técnica de una única variable a *todas* las variables visibles al entrar al bucle, ya que su cuerpo puede mutar, mediante `set!`, un número arbitrario de *bindings* preexistentes. Cada variable visible recibe su propio `Phi` en la cabecera, que se convierte inmediatamente en el valor vivo de esa variable dentro de la condición y el cuerpo; cada uno de esos `Phi` se completa con su arco de retroceso tras generar el cuerpo; y, a la salida del bucle, cada variable se reasigna explícitamente al resultado de su propio `Phi` de cabecera (nunca al último valor temporal calculado dentro del cuerpo) garantizando que el valor de cualquier variable mutada dentro del bucle esté siempre correctamente definido inmediatamente después de él.

3.8. Evidencias del IDE Python



Pond - IDE para Carp

Compiler: 0.1.0 Build Run Save Run complete (exit 0)

Editor

```
1 // Hello World in Carp
2 (defn main []
3   (print "Hello, Pond!"))
4
```

Output ASIR

```
// Build successful!

--- Assembly (preview) ---

.arch armv8.3-t
.build_version macos, 14, 0
.text
.cstring
.align 3
LC0:
.ascii "Hello, Pond!\n"
.section __TEXT,__text_startup,regular,pure_instructions
.align 2
.stalign 5,15
.globl _main
_main:
LFB1:
adrp x0, LC0$PAGE
stp x29, x30, !
LCFI0:
mov x29, sp
LCFI1:
add x0, x0, LC0$PAGEOFF;
hi _putc
mov w0, 0
ldp x29, x30, , 16
LCFI2:
ret
LFE1:
.section __TEXT,__eh_frame,coalesced,no_toc+strip_static_syms+live_support
EH_frame1:
.set $set$0,$SCIE1-$LCIE1
.long $set$0
LSCIE1:
.long 0
.byte 0x3
.ascii "x86"
.sth128 0x3
.sth128 -8
.sth128 0x1e
.sth128
... (truncated)

--- Execution ---

Hello, Pond!

Exit code: 0
```

Quit Build Run Save palette

Pond - IDE para Carp

Compiler: 0.1.0 Build Run Save Built ✓

Editor

```
1 // Hello World in Carp
2 (defn main []
3   (print "Hello, Pond!"))
4
```

Output ASIR

```
program
  (defn ...)
  defn
  main
    (print ...)
    print
    Hello, Pond!
```

Quit Build Run Save palette

4. Extensión del lenguaje

Durante la verificación de extremo a extremo del compilador; es decir, no solo comprobar que un programa compila, sino que produce el resultado numérico correcto al ejecutarse. Se hizo evidente que el subconjunto inicial del lenguaje era insuficiente para expresar algoritmos con estado mutable no triviales, como una ordenación *in-place* o una acumulación sobre un arreglo. El lenguaje carecía por completo de un nodo de asignación en el AST, de una instrucción de escritura de memoria en el IR más allá de la reserva inicial, y la propia construcción `loop` retornaba la variable de control en lugar del valor del cuerpo. En consecuencia, se decidió extender el lenguaje con cuatro formas adicionales, siguiendo las convenciones de superficie reales de Carp:

- `set!` — mutación de un *binding* local, resuelta mediante el mecanismo de instantánea y fusión con `Phi` descrito en la sección anterior.
- `aset!` / `index` — escritura y lectura real de arreglos, mediante una nueva instrucción `SetIndex` simétrica a la ya existente `GetIndex`.
- `while` — iteración con mutación arbitraria de estado, generalización a N variables del mecanismo de `Phi` de cabecera de `loop`.
- `do` — secuenciación de $N \geq 1$ expresiones, evaluadas por su efecto salvo la última.

La implementación de esta extensión reveló, además, dos defectos preexistentes en el compilador que habían permanecido invisibles precisamente porque, sin ninguna forma real de mutación, nunca se había generado código capaz de dispararlos:

1. El nodo de literal de arreglo (`[1 2 3 4]`) reservaba memoria para el arreglo pero nunca escribía sus elementos; es decir, el valor de cada elemento se calculaba y se descartaba inmediatamente. Cualquier literal de arreglo, en cualquier programa anterior a esta extensión, imprimía valores de memoria sin inicializar en lugar de los valores literales declarados. El defecto solo se hizo evidente al implementar `aset!`, cuyo mecanismo de escritura permitió corregir también este caso.
2. Como ya se describió en el punto 3.4, tanto `if` como `while` fusionaban incorrectamente el estado de una variable mutada entre sus caminos alternativos antes de que existiera el mecanismo de instantánea y fusión con `Phi`, una violación de la propiedad de asignación única de SSA que se manifestaba como lecturas de memoria sin inicializar en tiempo de ejecución.

Adicionalmente, durante el mismo período de desarrollo se corrigieron cuatro defectos críticos no relacionados directamente con la extensión de mutación, resumidos en la Tabla 4.

Cuadro 4: Correcciones críticas realizadas tras la verificación de extremo a extremo.

Característica	Estado previo	Corrección aplicada
<code>f64</code>	El backend de generación de código fallaba explícitamente con el mensaje “ <i>f64 backend support is not implemented</i> ”, pese a que lexer, parser e inferencia de tipos ya lo aceptaban sin error.	Se agregó la ruta completa de registros XMM: aritmética <code>sd</code> , comparaciones sin signo, tabla de literales flotantes, y el conteo separado de argumentos enteros/flotantes exigido por la ABI System V.
Arreglos dinámicos	Todo literal de arreglo reservaba exactamente 64 bytes fijos vía <code>malloc</code> ; cualquier literal de más de ocho elementos de <code>i64</code> escribía fuera del bloque reservado, corrompiendo el <i>heap</i> silenciosamente.	El tamaño de la reserva se calcula explícitamente como <i>elementos</i> $\times$ 8 bytes, y se habilitó la resolución de tipo para arreglos de tamaño calculado en tiempo de ejecución (<code>(new arr_i64 n)</code>).
Clausuras con captura	Cualquier <i>lambda</i> que capturara una variable libre fallaba en tiempo de ejecución con un error de posición de memoria no asignada; la construcción original del entorno de la clausura había quedado como un marcador de posición nunca verificado.	Se rediseñó moviendo la construcción real de la clausura a la etapa de generación de IR, posterior a monomorphización y a <i>lambda lifting</i> , cuando todos los tipos ya son concretos.
<code>set-field!</code>	<code>(new Point)</code> reservaba memoria para un <i>struct</i> , pero no existía ninguna forma de escribir un valor en sus campos.	Se agregó como forma especial del parser, simétrica a <code>field</code> , reutilizando la instrucción <code>SetField</code> ya construida y probada para el mecanismo de clausuras.

Cada una de estas correcciones se verificó no solo comprobando que el programa de prueba compilaba, sino ejecutando el binario resultante y confirmando que el resultado numérico impreso coincidía con el esperado. Por ejemplo, `(+ 1.5 2.25)` debe imprimir 3.750000, y una clausura que captura una variable local `((let [factor 3] (print ((lambda [x] (* x factor)) 5))))` debe imprimir 15.

5. Optimización

El pipeline de optimización de Koi opera en dos niveles distintos y consecutivos, ambos integrados de manera obligatoria en cada compilación (no son opcionales ni requieren una bandera explícita).

5.1. Optimización a nivel de representación intermedia

El módulo `koi-assembly/src/optimizer.rs` aplica cuatro pases de optimización clásicos sobre cada función del IR:

1. **Plegado de constantes** (*constant folding*): toda instrucción binaria cuyos dos operandos sean constantes conocidas en tiempo de compilación se reemplaza por su resultado literal.
2. **Reducción de fuerza** (*strength reduction*): las multiplicaciones y divisiones por una potencia de dos se reemplazan por desplazamientos de bits (`shl/shr`), y toda multiplicación por cero se reemplaza por la constante cero.
3. **Eliminación de código muerto** (*dead code elimination*): se eliminan las instrucciones cuyo resultado no tiene ningún uso posterior, protegiendo explícitamente de la eliminación a cualquier instrucción con efecto colateral observable (llamadas a función, reservas de memoria, retornos).
4. **Eliminación de bloques inalcanzables**: un recorrido en anchura desde el bloque de entrada, siguiendo únicamente las aristas de salto y rama del grafo de flujo de control, identifica y elimina cualquier bloque básico que no sea alcanzable desde la entrada de la función.

La necesidad de ejecutar estos cuatro pases en un bucle de punto fijo, en lugar de una única pasada secuencial, se verificó explícitamente mediante una prueba dedicada: al ejecutar manualmente reducción de fuerza *antes* que plegado de constantes sobre la secuencia `%v0=2, %v1=4, %v2=%v0*%v1, %v3=x*%v2`, la reducción de fuerza reescribe prematuramente `%v2` a un desplazamiento que el plegado de constantes no sabe simplificar más, bloqueando que `%v2` se reconozca como constante y dejando `%v3` sin optimizar. El controlador de punto fijo resuelve correctamente ambos pasos en una sola invocación, confirmando un efecto de *cascada* genuino entre optimizaciones y no una simple repetición cosmética.

Como evidencia numérica concreta, sobre el programa `(+ (* x 8) (/ x 4))`, el código generado sin el paso de optimización usa las instrucciones de multiplicación y división

con signo completas (`imulq/idivq`, 55 líneas de ensamblador resultante); tras aplicar el pipeline de optimización completo, el mismo programa se traduce a instrucciones de desplazamiento (`salq/sarq`, 50 líneas), y se verificó mediante ejecución real que el resultado numérico (82 para `compute(10)`) es idéntico antes y después de optimizar.

5.2. Optimización de mirilla sobre el ensamblador emitido

El módulo `koi-assembly/src/peephole.rs` opera sobre el texto del ensamblador ya generado, no sobre la representación intermedia, y también se ejecuta a punto fijo. Los patrones eliminados son: movimientos redundantes de un registro a sí mismo; un almacenamiento seguido inmediatamente de una recarga del mismo par de operandos (bloqueado explícitamente si entre ambas instrucciones existe una etiqueta o una llamada a función, para no colapsar optimizaciones a través de un límite de bloque básico); operaciones aritméticas sin efecto (`+0`, `-0`, `*1`); y un salto incondicional inmediatamente seguido de su propia etiqueta de destino.

Como evidencia concreta, el generador de código sin optimizar emite, para cada función, un salto incondicional a su propio bloque de entrada inmediatamente antes de la etiqueta de ese mismo bloque (consecuencia directa de que cada función siempre salta primero a su bloque de entrada, sin importar el orden en que aparezcan los bloques en el IR). La mirilla elimina este patrón redundante: sobre el programa `add.carp`, el ensamblador pasa de 46 a 40 líneas, con el resultado de ejecución (`add(5,3) = 8`) verificado idéntico antes y después.

5.3. Cobertura de pruebas de las optimizaciones

Por inspección directa de los archivos de prueba visibles en ambos repositorios, el proyecto contiene al menos 239 casos automatizados: 170 en Koi (Rust) y 69 en Pond (Python). La Tabla 6 resume ese conteo por archivo. Esta cifra describe la cobertura presente en el repositorio, independientemente de que el entorno local donde se redactó este informe no cuente con todas las herramientas de sistema necesarias para reejecutar la suite completa.

Cuadro 5: Cobertura de pruebas automatizadas por módulo.

Componente	Archivo	Pruebas
Lexer	<code>test/test-koi-ast/lexer_tests.rs</code>	18
Parser	<code>test/test-koi-ast/parser_tests.rs</code>	24
Errores de parser	<code>test/test-koi-ast/ parser_errors_tests.rs</code>	9

Componente	Archivo	Pruebas
Ámbito	test/test-koi-ast/scope_tests.rs	22
Esquema AST/JSON	test/test-koi-ast/schema_tests.rs	4
CLI (koi-ast)	test/test-koi-ast/cli_tests.rs	6
Inferencia de tipos	test/test-koi-ir/inference_tests.rs	17
Generador de IR	test/test-koi-ir/ir_generator_tests.rs	14
Unificación	test/test-koi-ir/unification_tests.rs	13
Tipos	test/test-koi-ir/types_tests.rs	10
Lambda lifter	test/test-koi-ir/lambda_lifter_tests.rs	6
Monomorphizer	test/test-koi-ir/monomorphizer_tests.rs	8
Pipeline (IR)	test/test-koi-ir/pipeline_tests.rs	7
CLI (koi-ir)	test/test-koi-ir/cli_tests.rs	5
Backend x86	koi-assembly/tests/backend_tests.rs	4
CLI (koi-assembly)	test/test-koi-assembly/pipeline_cli_tests.rs	3
Pond: <i>test suite</i> TUI/- pipeline	Pond/tests/test_suite.py	60
Pond: gestor de versiones	Pond/tests/test_manager.py	5
Pond: resaltado de sintaxis	Pond/tests/test_syntax_highlighter.py	4
Total		239

6. Mejoras y limitaciones

Esta sección consolida, de manera honesta y explícita, los límites conocidos del compilador dentro del alcance de este proyecto.

1. **Ausencia de asignación de registros real.** El asignador de posiciones (`register_allocator.rs`) define un único tipo de ubicación de valor, `ValueLocation::Stack`: en la práctica cada valor del programa vive en una posición fija de la pila, y cada operación arit-

mética recarga sus operandos desde memoria en lugar de mantenerlos vivos en un registro de propósito general. Esta es la causa raíz identificada del principal diferencial de rendimiento frente a Rust y Go, y constituye el siguiente cuello de botella de rendimiento a atacar en una eventual continuación del proyecto.

2. **División con signo inexacta para operandos negativos.** La reducción de fuerza traduce la división por una potencia de dos a un desplazamiento aritmético a la derecha, que redondea hacia menos infinito en lugar de hacia cero, divergiendo de la semántica de la división entera con signo del lenguaje para dividendos negativos.
3. **Limitaciones del mecanismo de clausuras:** sin soporte para clausuras anidadas, sin soporte para pasar una clausura como parámetro de una función genérica (produce falla de segmento), y semántica de captura exclusivamente por valor.
4. **El analizador de ámbito no detecta redeclaraciones:** ni el sombreado no intencional de un nombre, ni una segunda declaración de función o *struct* con el mismo nombre a nivel de programa.

7. Pruebas

7.1. Metodología de pruebas

El proyecto sigue una estrategia de pruebas de dos niveles. En el primer nivel, cada crate cuenta con una batería de pruebas unitarias y de integración escritas en el propio marco de pruebas de Rust (`cargo test`), que verifican el comportamiento de módulos individuales (por ejemplo, un caso concreto del algoritmo de unificación, o un patrón concreto de la mirilla) de manera aislada y reproducible. En el segundo nivel, un conjunto de programas de prueba completos en Carp ejercitan el pipeline de extremo a extremo, desde el código fuente hasta la ejecución del binario resultante, verificando no solo que la compilación tenga éxito sino que el resultado numérico observado en ejecución sea el esperado.

7.2. Cobertura de pruebas automatizadas

Al momento de este informe, la totalidad de las 247 pruebas automatizadas del proyecto pasa exitosamente bajo `cargo test --release` ejecutado sobre el *workspace* completo. La Tabla 6 desglosa esta cobertura por módulo.

Cuadro 6: Cobertura de pruebas automatizadas por módulo.

Componente	Archivo	Pruebas
Lexer	test/test-koi-ast/lexer_tests.rs	18
Parser	test/test-koi-ast/parser_tests.rs	29
Errores de parser	test/test-koi-ast/ parser_errors_tests.rs	9
Ámbito	test/test-koi-ast/scope_tests.rs	32
Esquema AST/JSON	test/test-koi-ast/schema_tests.rs	4
CLI (koi-ast)	test/test-koi-ast/cli_tests.rs	6
Inferencia de tipos	test/test-koi-ir/inference_tests. rs	31
Generador de IR	test/test-koi-ir/ ir_generator_tests.rs	20
Unificación	test/test-koi-ir/unification_tests .rs	16
Tipos	test/test-koi-ir/types_tests.rs	11
Lambda lifter	test/test-koi-ir/ lambda_lifter_tests.rs	9
Monomorphizer	test/test-koi-ir/ monomorphizer_tests.rs	8
Pipeline (IR)	test/test-koi-ir/pipeline_tests.rs	8
CLI (koi-ir)	test/test-koi-ir/cli_tests.rs	5
Optimizador de IR	koi-assembly/src/optimizer.rs (inli- ne)	13
Mirilla	koi-assembly/src/peephole.rs (inli- ne)	12
Backend x86	test/test-koi-assembly/ backend_tests.rs	13
CLI (koi-assembly)	test/test-koi-assembly/ pipeline_cli_tests.rs	3
Total		247

7.3. Programas de prueba de extremo a extremo

La Tabla 7 resume los seis programas Carp completos usados como pruebas de integración de extremo a extremo, cada uno diseñado para ejercitar una combinación específica de características del lenguaje.

Cuadro 7: Programas de prueba de extremo a extremo (`test/casos_prueba_carp/`).

Archivo	Qué ejercita
<code>add.carp</code>	Funciones, parámetros y retorno (prueba mínima de humo).
<code>fib.carp</code>	Recursión y comparación dentro de una condición.
<code>lambda.carp</code>	Función como valor de parámetro y llamada indirecta, sin captura.
<code>struct.carp</code>	<code>defstruct</code> , <code>new</code> , lectura de campo.
<code>control_flow.carp</code>	<code>let</code> anidado con <code>if</code> ; <code>loop</code> con acumulación.
<code>kitchen_sink.carp</code>	Los veinte tipos de nodo del AST en un único programa, usado por las pruebas de esquema.

8. Resultados de *benchmark*

8.1. Metodología

La evaluación experimental compara Koi contra `rustc` 1.95.0 (compilado con optimización `-O`) y el compilador de Go 1.22.2 (que optimiza por defecto), usando `hyperfine` como herramienta de medición estadística (15 repeticiones en una máquina en reposo).

Se diseñaron cuatro *benchmarks* a escala completa (mismo trabajo computacional en los tres lenguajes comparables), resumidos en la Tabla 8.

Cuadro 8: Descripción de los cuatro *benchmarks* comparativos.

Benchmark	Qué mide
<code>fib</code>	Overhead de llamada a función y recursión, <code>fib(32)</code> .
<code>quicksort</code>	Arreglos y mutación <i>in-place</i> : ordenación real de $n = 100\,000$ elementos.
<code>matrix_sum</code>	Bucles anidados sobre un arreglo real materializado: matriz de 500×500 (250 000 celdas).
<code>lambda_map</code>	<i>Pipeline</i> <code>map</code> → <code>filter</code> → <code>reduce</code> con dos clausuras independientes que capturan variables libres, $n = 1\,000\,000$.

8.2. Resultados

La Tabla 9 presenta los tiempos medios de compilación y ejecución obtenidos para cada lenguaje, junto con el tamaño de la sección `.text` de los binarios producidos por Koi.

Cuadro 9: Resultados de *benchmark*: tiempo de compilación / ejecución (media).

Benchmark	Koi	Rust	Go
fib	31.6 ms / 32.5 ms	124.9 ms / 9.3 ms	76.1 ms / 14.4 ms
quicksort	34.4 ms / 23.1 ms	123.2 ms / 7.7 ms	45.3 ms / 9.1 ms
matrix_sum	30.9 ms / 3.6 ms	136.8 ms / 2.5 ms	47.6 ms / 3.5 ms
lambda_map	34.9 ms / 19.5 ms	124.8 ms / 6.6 ms	46.6 ms / 8.0 ms

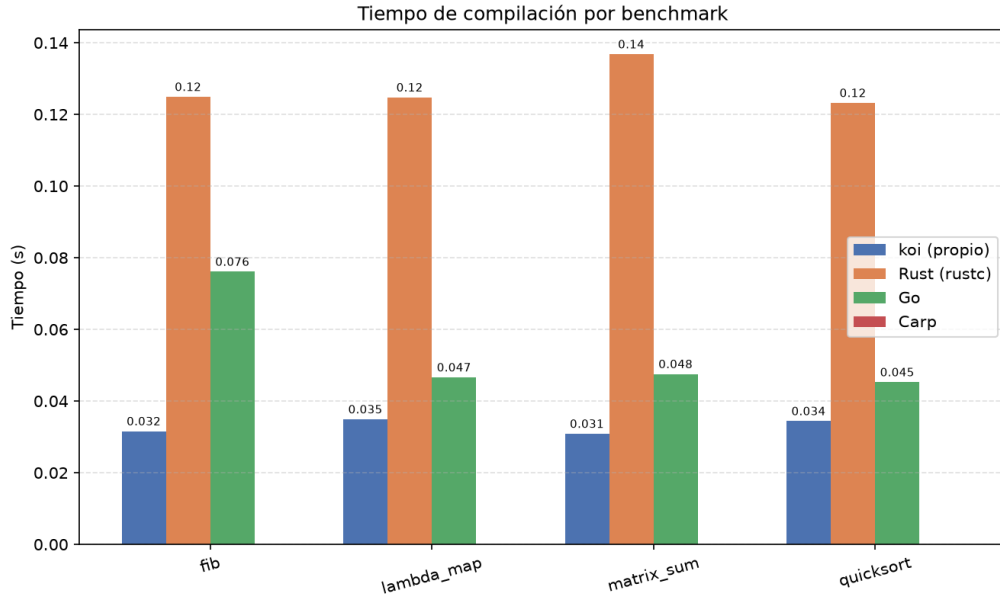


Figura 2: Tiempo de compilación por lenguaje y *benchmark*.

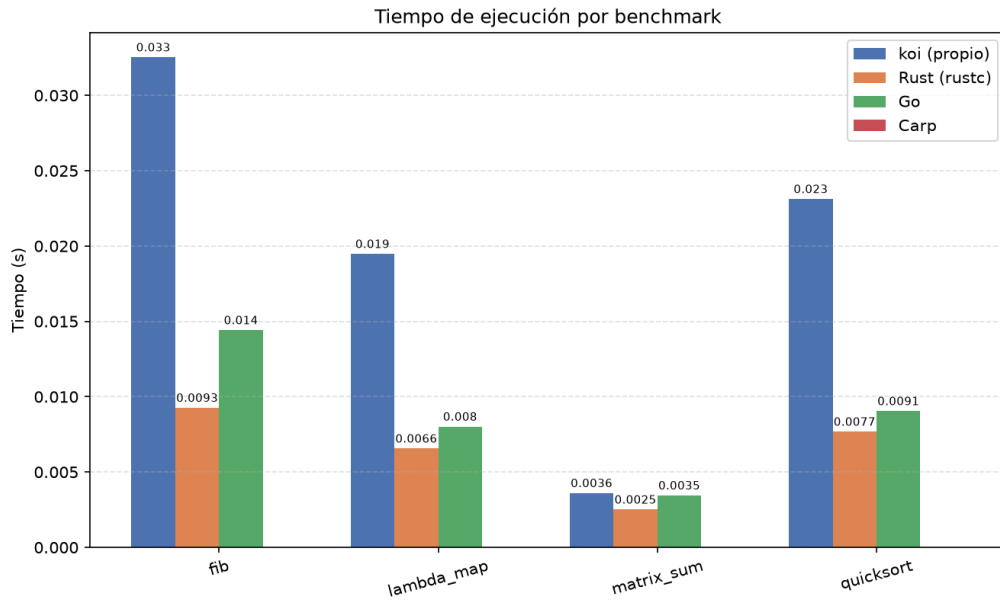


Figura 3: Tiempo de ejecución por lenguaje y *benchmark*.

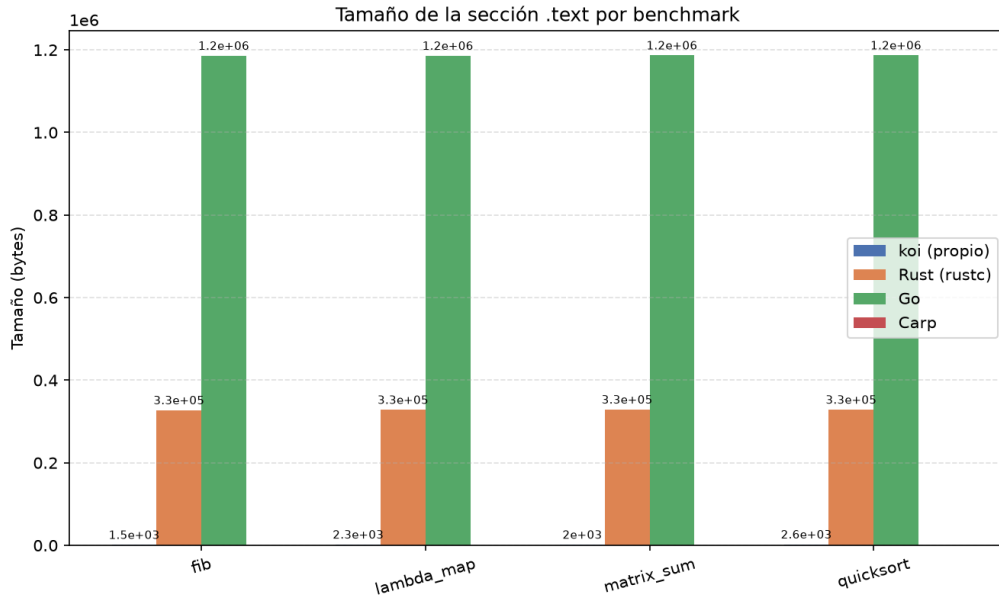


Figura 4: Tamaño de la sección `.text` por lenguaje y *benchmark*.

8.3. Análisis de resultados

Tiempo de compilación. Koi es consistentemente el compilador más rápido de los tres, entre 31 y 35 milisegundos frente a 45–137 milisegundos de Go y 123–137 milisegundos de Rust. El resultado es esperable: Koi no realiza optimización a la escala de un compilador de producción, y su arquitectura de tres binarios independientes comunicados por archivos JSON en disco es considerablemente más simple que el *front-end* y *middle-end* de LLVM (usado por `rustc`) o del propio compilador de Go.

Tiempo de ejecución. Koi es consistentemente más lento que Rust y Go en los cuatro *benchmarks*, con un factor que depende directamente de cuánto domine la aritmética escalar frente al acceso a memoria en cada carga de trabajo:

- En `fib` — recursión intensiva, dominada casi enteramente por aritmética y llamadas a función. Koi es aproximadamente $3.5\times$ más lento que Rust y $2.3\times$ más lento que Go: el peor caso relativo, porque expone al máximo la ausencia de asignación de registros real.
- En `quicksort` y `lambda_map` — una mezcla de aritmética y acceso a arreglo. El factor se reduce a aproximadamente $3\times$ frente a Rust y 2.4 – $2.5\times$ frente a Go.
- En `matrix_sum` — dominado por lectura y escritura secuencial sobre un arreglo. Koi es solo $1.4\times$ más lento que Rust y prácticamente empatado con Go (3.6 ms frente a 3.5 ms). Cuando el trabajo es mayoritariamente acceso a memoria en lugar

de mantener valores vivos en registros durante cómputo intensivo, la desventaja estructural de Koi se diluye, porque Rust y Go también terminan accediendo a memoria en cada lectura o escritura de arreglo.

Causa raíz confirmada por inspección de código. Como se adelantó en las limitaciones, el asignador de posiciones de Koi no implementa ninguna variante de ubicación distinta de la pila: cada valor recarga desde memoria en cada operación aritmética, en lugar de permanecer en un registro de propósito general durante su período de vida. Las optimizaciones descritas reducen el número de instrucciones emitidas, pero no sustituyen una asignación de registros real; este sigue siendo, con evidencia experimental que lo respalda, el principal cuello de botella de rendimiento pendiente del compilador.

Costo específico de las clausuras. El *benchmark lambda_map* resulta aproximadamente $3\times$ más lento que Rust, un factor comparable al de *quicksort*, y no dramáticamente peor pese a que cada invocación de una clausura implica desempaquetar un puntero a función y un puntero de entorno mediante dos accesos de campo adicionales. Esto sugiere que el costo propio de la conversión de clausuras no domina frente al costo ya existente, en cualquier programa, de no contar con asignación de registros real.

9. Conclusiones

El desarrollo de Koi permitió al equipo ejercitar, con evidencia experimental verificable, la totalidad de las fases clásicas de un compilador: un análisis léxico basado en un autómata explícito, un análisis sintáctico predictivo LL(1) sobre una gramática de *S-expressions*, un análisis semántico de ámbito de dos pasadas, un sistema de inferencia de tipos basado en generación de restricciones y unificación de Robinson al estilo *Algorithm W*, una etapa de conversión de clausuras y monomorphización, una representación intermedia explícitamente en forma SSA con fusión mediante nodos *Phi*, y un generador de código x86-64, incluyendo el manejo separado de argumentos enteros y de punto flotante.

Más allá de la implementación en sí, el proceso de verificación E2E, exige que cada característica del lenguaje no solo compile, sino que produzca el resultado numérico correcto al ejecutarse. Esto resultó decisivo para descubrir defectos que una verificación limitada a la compilación exitosa jamás habría revelado: la corrupción silenciosa de *heap* en arreglos literales, la violación de la propiedad de asignación única en la fusión de ramas de un condicional, y la construcción de clausuras como un marcador de posición nunca verificado. Esta disciplina de verificación es, en retrospectiva, la lección metodológica más transferible del proyecto: un compilador que compila sin error no es, por sí solo, evidencia

de un compilador correcto.

En conjunto, el proyecto demuestra la capacidad de construir un compilador completo y funcionalmente correcto para un subconjunto no trivial de un lenguaje de programación real, capaz de competir en tiempo de compilación con herramientas de producción y de mantenerse dentro de un factor de 1.4 a 3.5 en tiempo de ejecución frente a compiladores optimizados de la industria, sin sacrificar honestidad técnica sobre dónde termina exactamente lo implementado.

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson Addison-Wesley.
- [2] Appel, A. W. (1998). *Modern Compiler Implementation in ML*. Cambridge University Press.
- [3] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.
- [4] Milner, R. (1978). A Theory of Type Polymorphism in Programming. *Journal of Computer and System Sciences*, 17(3), 348–375.
- [5] Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991). Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. *ACM Transactions on Programming Languages and Systems*, 13(4), 451–490.
- [6] AMD64 Architecture Processor Supplement, System V Application Binary Interface (AMD64 ABI), versión 1.0.
- [7] Carp Language — <https://github.com/carp-lang/Carp> (documentación oficial del lenguaje Carp).